ॐ

DOMS Page No.
Date / /

OOPS – Object Oriented Programming

→ OOP is faster and easier to execute.
→ OOP provides a clear structure for the programs.
→ OOP helps to keep the JAVA code DRY "Dont Repeat Yourself", and makes code easier to maintain, modify and debug.
→ OOP makes it possible to create full reusable applications with less code and shorter development time.

Class :- A blueprint or template for creating objects. → Defines a set of attributes (variable) and behaviour (methods)
↳ defines what object will contain and do.
↳ logical construct

```
class Student {
    int rno;
    int name;
    int marks;
    static int vary=4;
}
```

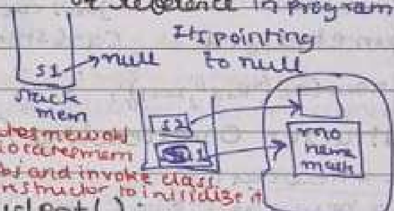
Object :- instance of class → when you create object, you are creating a real-world entity based on the blueprint provided by the class.
It holds specific data & can perform actions (through methods)
→ physical construct // Occupies space in memory

3 properties :
① state → refers to values of its attributes (eg. rno, name, marks)
② identity → every obj has unique identity
identity distinguishes one obj from another, even if same class and state
↳ represented by memory address or reference in programming
③ behaviour → refers to methods or function that object can perform

reference to type Student

Student s1; // Declare → It's pointing to null

s1 = new Student()



new: dynamically allocates memory & returns a reference to it
creates new obj & allocates mem for obj and invoke class constructor to initialize it

Student s1 = new Student();
Compile time | runtime

** You cannot make a reference variable point to an arbitrary memory location → ensures safety, portability, security, automatic mem mng

memory safety
↓
prevents dangling pointers (where a pointer refers to mem loc that has been freed)
memory leaks

binds instance methods/variables with reference variables
dot
Student.vary
s1.marks = 15;
access instance members
access static members → can be func too

JVM hides specific details like mem add and hardware specific
↓
hence not depend on specific mem layout

Java ensures that obj in one program can't overwrite or interfere with mem of other prog ing
↓
Isolation
static

Memory :-

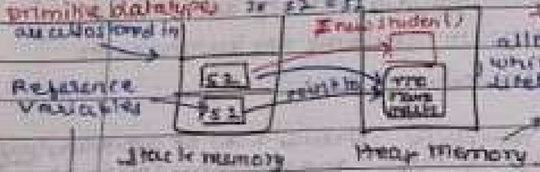
When programs compile,

Classes → they are loaded in memory

↳ loaded into Method Area / MetaSpace

stores class defⁿ, static (var & methods), bytecode of class

primitive datatypes are also stored in



Heap memory:
- suitable for long-lived entity
- allows for dynamic & flexible management which is essential for obj whose sizes & lifetimes can vary during runtime
- Large pool of memory used for dynamic memory allocation in Java
- can grow & shrink dynamically

Reference variables:
- hold memory address of object rather than object itself
- it points to an obj in heap memory
- stored in stack memory

Reason: Stack is faster and has LIFO structure — means reference variable memory is allocated & deallocated quickly when methods are called. They are short-lived & exist only during exe of method or block of code and return.

* Constructors

special method in class that is called when an object of that class is created. Purpose: initialize the object

→ same name as class

→ no return type (not even void)

① Default : `Car c1 = new Car();`
- For primitive data types: 0, false, null (boolean)
- For object type: null (for obj & String)

↳ you may / may not define it

↳ you can define it by :-

```
public Car(){
this.color = "Black";
this.year = 2020;
```

→ **this keyword**
1. refer to instance variables
2. call other constructor
   ```
   public Car() {
     this("Black", 2020);
   }
   ```
3. pass current object as parameter
4. Access curr obj method

② Parameterized : `Car(String color, int year) { ... }`

`Car c3 = new Car("Red", 2015);`

③ Constructor Overloading
- diff parameter type or number
- no return type
- no inheritance

```
Car c1 = new Car();
Car c2 = new Car("Black", 2021);
```

Allows → Different initialization (Flexibility), Cleaner code, Enhanced Readability

④ Copy Constructor : create a new object by copying the data of an existing object of the same class

```
Box(Box other)
{ this.l = other.l;
} this.h = other.h;
```

→ Used to make seperate, independent copy of an object.

---

java.lang package → Integer, Boolean, Long, Double, Character

☐ Wrapper Classes : used to wrap/encapsulate the primitive data types into object.

Why Use → ① Object Requirements : Some Java features req obj not primitive. Like ArrayList, HashMap can't store primitive directly. They req obj

② Utility Methods : useful methods like parsing or conv from one type to another

③ Autoboxing and Unboxing

automatic conversion from primitive to wrapper class; reverse

```
Integer a = new Integer(42);
Integer b = 45;
```
equivalent

```
void swap(Integer a, Integer b)
{
 ...
}
```
→ wont swap → Why? (ent it like obj, reference variable)

Reason: Integer is final class

* final keyword :

`final int a = 2;` → now it cant be modify → if u try it will give error saying u cant modify a final variable

↳ They have to be initialized (always) while declaring.

"This immutability holds true for only primitive datatype" (that it can't change value)

```
final Student s1 = new Student();
s1.name = "new_name"; ✓
s1 = other object ✗ ≈ s1 = new A(); ✗
```

① Final variables : value cant change for primitive

↳ In reference var, you cant change reference itself (i.e object it points to), but you can modify its internal state

② Final Methods : prevents method from being overridden by any subclass

↳ Ensure behaviour of method is fixed and cant be altered by ↗

↳ cause compile-time error if tried in subclass (extends)

③ Final Classes : prevent class from being inherited

④ Final Parameters : parameter value cannot be reassigned inside the method

```
modifValue(final int a)
{
  // a = 10 // This would cause compile-time error
  System.out.println(a);
}
```

☐ Garbage Collector : process of automatically reclaiming memory by destroying objects that are no longer in use (not reachable)

If obj is Unreachable (no live ref pointing to it) → Mark & Sweep Algo
- Mark: it marks all obj that are reachable from active references
- Sweep: removes all that are not marked by sweeping through heap

⇒ Heap :
① Young Generation — new obj are 1st created in ↗ & if they survive long enough promoted to
② Old Generation
③ Permanent Generation (replaced by MetaSpace) — fixed; MetaSpace: dynamic, not part of heap

Finilization → b4 obj is destroyed, finalizer method can be called → gives obj chance to perform cleanup ops, b4 they are removed

JOMS | Page No.
Date / /

Override finalize() method

A obj; for i = 0 to 5 { obj = new A(); } — class A { protected void finalize() throws Throwable { S.o.pln("destroyed"); } }

← prints 5 times

Not preferred :- Non-deterministic, slower, Deprecated

PACKAGE :- a namespace and container for related classes and interfaces

↳ grouping + logical boundary + access (visibility) control + namespace management

Think of it as a folder

- Data Encapsulation → within a class
- Behaviour " → through interface
- Module " → through packages

finance.Account
hr.Account
<package-name>.<class-name>

package com.company.project.module; → Declaration
hierarchical structure (like folder)

① How does Java know which package a file belongs to?
↳ 1) package statement inside the file → If it can't find the one specified in the statement → compile error
2) directory structure on disk

② What happens when you compile a file? eg. package com.school; public class Maths { }
↳ javac Maths.java
(creates a .class file that remembers its name with the com.school.Maths) → checks com/school/Maths.java; if ✗ → error

∴ If u wanna use methods of Maths class in different class of diff package → just import it

* Import statement → does NOT load the class, does not bring file into memory

↳ It's a compile-time name resolution tool

Compiler sees import java.util.ArrayList; // ArrayList → map to java.util.ArrayList

Hence: java.util.ArrayList list = new java.util.ArrayList → We don't need this

If import: ArrayList list = new ArrayList();

↳ note: 2 classes with same name value ✗ → import mypkg.ArrayList; → If u did, u should write whole sentence to avoid it

Need: ① Readability ② Maintainability ③ Avoiding Repetition ④ Reducing Typing Errors

→ Import lets you use encapsulation

import java.util.List; — don't care whether it's ArrayList or LinkedList

Types of import:

① Single-Class Import
import java.util.ArrayList; → one specific class → clear & explicit

② Package (wildcard) Import
import java.util.*; → doesn't import subpackage
* imports all public classes in that package

③ Static import
import static java.lang.Math.PI;
you can use static members w/o class reference

JOMS | Page No.
Date / /

* Static : some property independent of objects → not directly related to obj

eg. Human class → name, age, married, population
name, age, married → diff obj have diff values; population → same for all objs

```
public class Human {
  int age; String name;
  boolean married;
  static long population;
  public Human( ... ){
    this.age = age;
    this.name = name;
    this.married = married;
    Human.population += 1;
  }
```

→ static variable : It can be accessed by any of its object of the class is being created and without referencing to that object

Human.population — You access it via class name

Still if you use this.population += 1; → still works → Reason: being let say I create obj 2, call constructor → see this.population hmm → It doesn't exist so it will check class → It is there → increment by 1 → now population = 1

Convention: So in human obj, now population = 1; obj 2 created, it doesn't exist but in class, yes

when you call static variable: Human.population — even though obj1.population will work

```
public static void main (String[] args) {
}
```

static → you can use this method without creating an object of that class

→ main func is first thing to ever run. Now imagine to run a func of class you need to create obj of class → how can u create a program to create obj for main as it is the first thing ever run → hence you should be able to run w/o creating an obj → hence it is static

* Static *

When you declare a member static :-

① It belongs to class, not to any particular object
② You can access it w/o creating an instance of the class
③ There is only one copy of static variable per class, shared by all objects

Static / Class Variables:
→ All obj share same copy
→ If one obj changes it, all other objs see the updated value
→ Initialized once when class is loaded

```
class Counter {
  static int count = 0;
  Counter() {
    count++;
  }
}
```

class Test
Counter c1 = new ...
c2 ...
c3 ...
S.o.pln(Counter.count)
this is 3

Static Methods:

```
class Calculator {
    static void greet() {
        print(); ✗
    }
    static int add(int a, int b)
    { return a+b; }
    void print() {
        S.o.Pln("Hello");
    }
    static void test() {
        S.o.Pln("Hi →", add(5,3));
    }
}
```

Cannot use non-static instance variable or methods directly

→ Can only access static members directly

```
class Test {
    public static void main()
    {
        S.o.Pln(Calculator.add(1,3));
        S.o.Pln(Calculator.print()); & Print
        S.o.Pln(Calculator.greet());
    }
```

can be called w/o creating objs

cannot do it

possible but error will give error

Reason → static fun^c are not dependent on objs.
we know that something that is not static, belongs to an object
∴ You can't call a non-static inside static because it's requiring an instance. But the fun^c you are using doesn't depend
→ way to do is explicitly mentioning object reference
like static void greet (Calculator c)
{ c.print(); } ✓

But vice-versa is true → Inside a non-static fun^c, you can call both static & non-static fun^c.

Static Block

→ Used to initialize static variables
→ Executes once when class is loaded, even before main()

```
class Example {
    static int data;
    static {
        data = 100;
        S.o.Pln("Block executed");
    }
}
```

main fun^c
S.o.Pln(Example.data);

O/P: Block executed
100

→ Executed exactly once
↓
when first obj is created i.e during loading

Static Nested Class

→ A class defined inside another class using static. Does not need an instance of outer class. Can access static members of outer class directly.

```
class Outer {
    static int x = 10;
    static class Inner {
        void show() {
            S.o.Pln("x = " + x);
        }
    }
}

Outer.Inner i = new Outer.Inner();
i.show(); // x=10;
```

Rule: No [this] keyword in static context
because this belongs to current object & static belongs to class

* Only Inner class can be static
Outside class cannot be static

Eg. Outer.Inner i = new Outer.Inner();

Now, it does not depend on Outer Object

```
public class InnerClasses {
    [static] class Test {
        String name;
        public Test(String name) {
            this.name = name;
        }
    }
    public static void main() {
        Test a = new Test("Shivam");
        Test b = new Test("Anush");
        S.o.Pln(a.name);   // Shivam
        S.o.Pln(b.name);   // Anush
    }
}
```

→ If not static, they belong to an instance of the outer class

Eg. Outer o = new Outer();
Outer.Inner i = o.new Inner();
↓
If we don't use static here
↓
Test → won't even run
(∵ main being static only call static methods)

Since Test is static
↓
we don't need to create obj of Outer class to run it

static means independent on outer class; it doesn't mean it can't have its own individuality

```
✗ static class A {
  }
```

→ is only meaningful inside a class
→ for outer class (not inside another class), so it doesn't have an outer instance anyway
↓
It's redundant & meaningless
(since it's already independent)

→ Since it itself is not dependent on any other class, so how can it be static

* All static members are resolved during compile time
↳ since they don't depend on object which is at runtime

Gives PrintStream obj

System.out.println("Hello, World");

- System → final class in java.lang
- out → static field in System
- println → method of PrintStream
- "Hello, World" → string literal → stored in String Pool

```
final
public class System {
    public static final PrintStream out;
       "      "     "   InputStream in;
}
```

⇒
```
public void println(String x)
{
    synchronized (this) {
        print(x);
        newLine();
    }
}
```

```
public String toString() {
    return getClass().getName() + "@" + Integer.toHexString(hashCode());
}
```

For eg:- com.company.Example@4627c284
(package name, class name)

Inner classes $ Test → if inner class

single "controller" object → eg. SystemClock

Allows only one instance of itself to be created

1. **Singleton Class**: Provides a global point of access to that instance

```
class Singleton {
    private Singleton() {
    }
    private static Singleton instance;
    public static Singleton getInstance() {
        if (instance == null) {
            instance = new Singleton();
        }
        return instance;
    }
}
```

Test Class

```
Singleton s1 = Singleton.getInstance();
   "      s2 =    "         "
S.O.Pln(s1 == s2);
```

true

Both reference point to same object

Problem: serialization creates a new obj → breaks singleton → soln: override readResolve() → return getInstance() (called by JVM during deserialization)

OOPS PRINCIPLES

① Inheritance ② Polymorphism ③ Encapsulation ④ Abstraction

1] Inheritance:

It is a mechanism by which one class (child / subclass) acquires / derives the properties and behaviour (fields & methods) of another class (parent / superclass)

Parent: Superclass / Base Class / Ancestor class — "Car IS A Vehicle"

↑ IS-A relationship — "code reusability"

Child: Subclass / Derived Class / Extended class → add new features or override existing ones

extends keyword:

```
class Child extends Base {
}
```

* When an object of a child class is created, the constructor of parent class MUST be executed first

↳ ∵ child obj contains parent part inside it → You can't build 2nd floor of a house before the foundation

parent super w, super h

← super() → constructor call to the immediate superclass

→ By default if you don't write → super() is inserted automatically at first line of child constructor

→ Parent has a parameterized constructor only

A: `A(int x) { }`

B: `B() { }`

Compile time error: super() is inserted but doesn't exist in parent class

Java does not automatically create a default constructor when any constructor is explicitly defined

but super(5); will work

"Object of Parent Class cannot use Child's attributes or methods"

Box: int l, b, h

↑

BoxWeight: int wt

It is the type of reference variable and not the type of object that determines what members can be accessed

```
Box b = new BoxWeight(2, 3, 4, 8)
S.O.Pln(b.l) ✓
S.O.Pln(b.wt) ✗
```

If h is private, it isn't accessible here also b.h ✗

```
BoxWeight bb = new Box(2, 3, 4); ✗
```

Reason: there are many variable in both parent & child class → You are given access to variables that are in the ref type i.e. BoxWeight → hence, you should have access to wt variable

↓

this also means that the ones you are trying to access should be initialized → but here the obj itself is of parent class, how will you call constructor of child → Hence error

* Every class has Object Class as a superclass → public Object() { } constructor

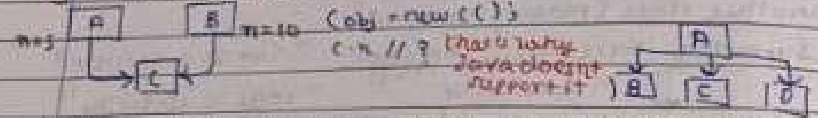
Advantage: You can handle many subclasses with one ref type
Parent pc = new Child();
↳ Using a superclass ref limits you to methods defined in super class
You cannot directly call subclass specific methods without casting

* Types of Inheritance:

1] Single Inheritance: One class extends another class

2] Multilevel Inheritance [A] ← [B] ← [C] — B extends A, C extends B
A has no idea of B & C. But C has idea of all above classes A & B

3] Multiple Inheritance: 1 class extending more than 1 class



4] Hierarchical Inheritance: 1 class inherited by many class

(not in Java) 5] Hybrid Inheritance: combination of single + multiple inheritance

* You can't inherit yourself: [A] → X → cyclic inheritance

2] Polymorphism: allows same method name or reference to behave differently depending on the object it refers to.
* many ways to represent

Language that possess class but no ability of polyⁿ: Object-based language
You can't advertise yourself as OOP if you don't have polyⁿ.

Polymorphism
- Compile Time / Static
  - Function Overloading — same method name but type, no. of arguments, return type, ordering can be different. eg: Multiple Constructors ↓ In Java, at compile-time it will decide which method to call → which method it can access → entire process called overriding
  - Operator Overloading
- RunTime / Dynamic
  - ↓ Method Overriding — achieved by inheritance. Child method will override Parent if child obj is used ↓ @Override // Annotation ↓ no error → if actually overriding; error → if not

Parent obj = new Child(); — which method will be called depends on
This is known as Upcasting & when ref variable is of superclass which particular method is called depends on object

Q] How Java determines which method to run?
→ For Overloaded methods: matching method signature
For Overriding methods:
↳ Dynamic Method Dispatch: It is what happens when Java chooses the correct version of the overridden method to call, based on actual object type at runtime. (Not ref type)

Note: When a method is declared final, Java does not allow subclasses to provide a new implementation of that method.
"Final prevents method overriding"
→ Preserves behaviour, Improve security, maintain consistency

Benefits: Better Performance → when a method is declared final, JVM knows this method cannot be overridden. Because of this guarantee, JVM does not need to check at runtime which version of method to call. Instead it can directly bind the method call at compile time (called static binding / Early binding). This reduces method call overhead and allows JVM to apply optimizations like inlining the method code, where the method's body is placed directly into the calling code. → Execution faster than non-final methods (require dynamic binding ↓ Late Binding) → more efficient

** Once you declared a class as Final, its method implicitly will become final too → Hence can't be inherited (you can't extend that class)

* Static methods can be inherited but cannot be overridden
Reason: Overriding works using runtime polyⁿ which depends on object. Static does not depend on object → they are bound at compile time and belong to class. Hence they can't be overridden.
→ If subclass defines static method with same signature → that method hiding, not overridden

* Access modifiers of overridden methods should be same or better. (It cannot be more restricted)

3] Encapsulation : wrapping up the implementation of the data members and the method inside a class
↳ wraps the code and data into a single entity or unit so that can be protected from outside world.
eg. Implementation issue (one will make the class, method, other has to implement the func)
Focus on internal (internal complexity)
→ Achieved using classes → Provides data hiding
→ Data is protected using access modifiers

4] Abstraction : hiding unnecessary details and showing only ~~valuable~~ information
eg. You need a key to start car (All mechanism behind you don't need to know)
eg. S.O.Pln( ) → Just print (don't care how internally are they working)
→ Design issue (Designer say this a abstract class they want → this - this but they write to)
Focus on external (what is visible to me)
→ Achieved using abstract classes and interfaces

* Data hiding is achieved via Encapsulation ~~& Encapsulation~~

□ Data Hiding : concept of restricting direct access to data so that it cannot be modified or ~~misused~~ from outside the class
↳ Achieved using private
↳ Data can be accessed only through controlled methods (getter/setter)
↳ Focus on security & protection of data

* ACCESS CONTROL :

Modifier	Class	Package	Subclass (same pkg)	Subclass (diff pkg)	World (diff pkg & not subclass)
public	✓	✓	✓	✓	✓
protected	✓	✓	✓	✓	
default	✓	✓	✓		
private	✓				

Q] When to use each access modifier ?
→ private : use for sensitive data to be protected → enforce controlled access via getter/setter → class variables, internal helper methods
default : when you don't want it to be accessed outside pkg → eg. student pkg you don't want it to accessed by some other pkg → used when class across a single module
protected : when subclass needs access → for methods meant to be extended or reuse → controlled inheritance
public : available everywhere
↳ main methods, API calls

* protected : In diff package, only subclass can access it → A extends B (it has protected num)
↳ now in class A you do
B obj = new B( ); ✗
∴ We A obj = new A(); ← Note class B doesn't know who extends it ← You can't print obj.num, You can't use this line anywhere in subclass
int n = obj.num;
↳ ∵ Only subclass knows what he has extended

Packages → User-defined
→ In-built

① lang : automatically imported → essential classes req for basic programming → Integer, String, Object, Class, Thread, Math, Runnable, StringBuilder
② io : i/o operations, File handling, reading and writing data, byte and char stream
eg. File, InputStream, OutputStream, FileReader, BufferReader, Writer
③ util : utility classes and data structures → Collection framework (ArrayList, HashMap) Date, Calendar, Scanner class, Random no. generator
④ applet : create java programs that run in browsers → (obsolete) → security & browser restriction
⑤ awt : gui, event handling → Frame, Button, Label, Graphics, Color
⑥ net : networking → internet connection, url handling → URL, SOCKET, ServerSocket

□ Object Class : - root of all classes in Java. Every class implicitly extends Object → java.lang.Object
methods :- ① toString () → Returns a string representation of obj → Classname@hashcode
② equals(Object obj) → Compares references (==) → content/value of objects (equals)
int x = 5 String x = new String ("hello") s1 == s2 False
int y = 5 String y = new String ("hello") s1.equals(s2) True
can be overridden
③ hashcode ()
final method ④ getClass() → Returns runtime class of object return type : Class <?>
String s = "Hello"; S.O.Pln(s.getClass()); → class java.lang.String
↳ .getSimpleName() .getName() .getPackage() .getSuperClass() .getMethods() → all public & inherited .getFields() → .getDeclaredFields() private too
⑤ clone () → returns copy of obj (shallow copy)
⑥ finalize () → called by Garbage Collector before destroying the obj. Used to release resources

* instanceof :- binary operator to test whether obj is an instance of a specific class or implements an interface

Animal ← extends Dog; Dog implements → Pet

Animal a = new Animal();
Dog d = new Dog();
Animal ad = new Dog();

a instanceof Dog } False
a instanceof Pet }
Rest all combinations → True

d, ad instanceof Pet } True
d, ad instanceof Dog }

Used for safe casting
```
if (ad instanceof Dog) {
    Dog d1 = (Dog) ad;
    d1.play(); → dog specific method
}
```

try : Wrap risky code throw : Explicitly throw exception
catch : Handle exception throws : Declare exception
finally : Always execute static int divide(a,b) throws ArithmeticException
if (b==0) {
throw new ArithmeticException("Invalid ...");
}
return a/b; // may throw an exception

```
try {
    int a = 10/0;
} catch (Exception e) {
    S.o.println(e.getMessage());
}
finally { S.o.println(" "); }
```

Custom Exception :- public class MyException extends Exception {
public MyException (String msg) {
super(msg); → sent to Exception class
} }

Object Cloning

→ It's a way to create a copy of an existing object in memory.

Human h = new Human(18, "Shivam");
Human twin = new Human(h); → clone but inefficient, we use new keyword to create

How cloning works?

① clone method defined in java.lang.Object ② By default, clone() provides shallow copy (just telling JVM we are cloning this)

③ To use it, a class must implement the Cloneable interface — marker interface, no methods, no fields. Tells Java to do something specific behaviour = by clone

1] Shallow Copy : Copies primitive data & obj ref, but not the objects they refer to.

```
class Person implements Cloneable {
    String name;   // immutable
    int[] scores;  // mutable obj (reference)
    Person(....) { }
    @Override
    protected Object clone() throws CloneNotSupportedException {
        return super.clone();
    }
}
```

Changes in referenced objects affect both original and clone.

use public → if want outside access

If a class doesn't implement Cloneable, calling clone() throws.

main fun →
```
int[] marks = {90,80};
Person p1 = new Person("Shivam", marks);
Person p2 = (Person) p1.clone();
p2.name = "Ravi";
p2.scores[0] = 100;
S.o.p (p1.name + " " + p1.scores[0]) // Shivam 100
S.o.p (p2 ... p2.scores[0]) // Ravi 100
```
may throw CloneNotSupportedException

2] Deep copy :- Copies everything including ref obj. Changes in clone don't affect the original.

```
Person(....) {
    this.scores = scores.clone(); // deepcopy
}
Object clone() throws .... {
    Person cloned = (Person) super.clone();
    cloned.scores = this.scores.clone(); // deepcopy
    return cloned;
}
```

* Virtualized method : methods decided at runtime based on actual object (dynamic binding)
↳ All non-static, non-final, non-private methods are by default virtual

Non-virtual method : compile time (static binding)
↳ static, final, private, constructors

□ Nested interface:

```java
public class A {
    public interface Nested {
        boolean isOdd(int num);
} }
```

(only public, private or protected)

```java
class B implements A.Nested {
    @Override
    public boolean isOdd(int num) {
        return (num & 1) == 1;
} }
```

But top-level interface has to be declared public or default
↳ private: accessible only inside enclosing class (there is no enclosing class)
protected: applies to inheritance (top-level interface don't belong to class hierarchy)

→ Nested interface inside interface are implicitly public & static

```java
interface Outer {
    interface Inner {
        void show();
} }
```

→ why? Interface cannot have instance members. Nested types must be accessible to implementers

* interface can extend other interface
* class extends class
* class implements interface

(interface, interface —extends→ interface)

□ Annotations:

They are special markers (metadata) added to code to give extra info to the compiler, tools, or frameworks w/o changing program logic.

"Annotation tell Java how to treat code, not what the code does"

Eg. @Override @Deprecated @SuppressWarnings

→ They are like an interface, but do not provide implementation

Defining:

```java
public @interface MyAnnotation {
    String desc() default "Default Desc";
    int version() default 1;
}
```

* **Comparable** is used when you want objects of a class to be compared with each other and sorted in a natural order.

```java
class Student implements Comparable<Student> {   // interface
    int id; String name; int marks;
    Student(",",") { ... }
    @Override
    public int compareTo(Student s) {
        return this.id - s.id;
    }
}
```

```java
List<Student> l = new ArrayList<>();
l.add(new Student(3, "ABC", 85));
l.add(                (2, "DEF", 30));
l.add(                (3, ...));
Collections.sort(l);   // uses compareTo
for (Student s : l)
    SOPln(s.id + " " + s.name);
```

* **Lamda Function**:- Short way to write anonymous func (func w/o name)
↳ Concise way to represent a single-method interface implementation
does not create a NEW method

Functional interfaces ← works only with this

(parameters) → expression / (parameters) → {statements}

```java
ArrayList<Integer> arr =
eg. arr.forEach((item) -> SOPln(item*2));
Collections.sort(l, (a,b) -> a-b);
```

```java
interface Operation {
    int operation(int a, int b)
}
```

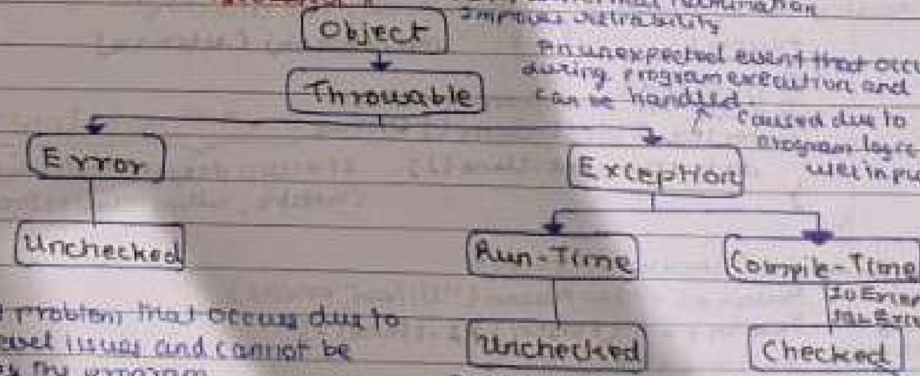
```java
Operation sum = (a,b) -> a+b;
Operation prod = (a,b) -> a*b;
Operation sub = (a,b) -> a-b;
LamdaFunctions calc = new LamdaFunctions();
SOPln(calc.operate(5, 3, sum));
     (calc.       (5, 3, prod));
     (calc.       (5, 3, sub));
```

In this class only make

```java
private int operate(int a, int b, Operation op) {
    return op.operation(a, b);
}
```

□ EXCEPTION HANDLING:-

program continues safely
prevents abnormal termination
improves reliability



Exception: An unexpected event that occurs during program execution and can be handled. Caused due to program logic or wrong input.

Compile-Time: IOException, SQLException

Error: A serious problem that occurs due to system-level issues and cannot be handled by the program
→ caused by JVM or system
→ Not recoverable
→ Program usually terminates
eg. OutOfMemoryError, StackOverflowError

Run-Time (Unchecked): Arithmetic, NullPointer, ArrayIndexOutOfBounds

Checked → Exception
FileReader fr = new FileReader("file...") (compile time)

* ABSTRACT CLASS

(You cannot create obj of abstract class)

→ A class that cannot be instantiated directly
→ may contain one or more abstract methods (methods w/o a body) and concrete methods (methods with a body) & fields/variables and constructors
→ It is designed to be subclassed, and its abstract methods **must be implemented** by the subclasses

* If class contains any one abstract method → must be declared abstract class

Reason → Abstract class may have abstract methods without any implementation → If we allow you to create an object of abstract class, JVM wouldn't know how to run them → no body → lead to incomplete objects with undefined behaviour

Similarly: No Abstract Constructors → But normal constructor is fine
Abstract static methods → no ✗ → static cannot be overridden, so having abstract static thing not possible
But just static method in abstract class is ok ✓
↳ Since they were never dependent on objects → you call Parent.hello(); (when Parent is abstract)

→ You can use abstract reference (runtime polym) → Parent son = new Son();
→ You can't have direct Abstract class → how can you implement when you can't inherit
* multiple Inheritance is still not allowed

* INTERFACES

→ It is an abstract type that defines a set of methods a class must implement.
→ It acts as a contract that specifies **what a class should do**, but not how it should do it (Used to achieve → Abstraction & Multiple Inheritance)
→ A class that implements interface must implement all methods of interface

1. All methods are public abstract by default (unless default/static)
2. All variables are public static final (constants)
3. Cannot create objects of interface → so var must be static → since no constructor then no initialization ∴ var = final

When to use a class?	When to use an interface?
→ when you need to represent a real-world entity with attributes & behaviour → when you need to create objects that hold state and perform activities → classes are used for defining templates for obj with specific functionality and properties.	→ when you need to define a contract for behaviour that multiple classes can implement → ideal for achieving abstraction & multiple Inheritance

Remember: Interfaces represent capabilities, not objects

Brake	Engine
start() stop()	start() stop() acc()

Car: @Override start(){ } ...

→ Why do 2 class don't collide here? → Because Car will anyway provide its own implementation anyway

helps in loose coupling → You can change implementation w/o changing the class (No forced change)

```
interface Payment {
    void pay(int amt);
}
class UPI implements Payment {
    public void pay(int amt) {
        S.O.Pln("Paid via UPI");
    }
}
class Card implements Payment
{
}

class Shop {
    Payment p;
    Shop(Payment p) {
        this.p = p;
    }
    void buyItem() {
        p.pay(100);
    }
}

main fun {
Shop s1 = new Shop(new UPI());
s1.buyItem(); // Paid via UPI
Shop s2 = new Shop(new Card());
s2.buyItem(); // Paid via Card
}
```

(Runtime poly^m at its best)

* In class inheritance, method calling follows a hierarchy-based lookup
starts from actual obj class → if method not found → moves upward through superclass chain
In interfaces, directly execute class method → ∵ implementing class is forced to provide method implementation
Even in default methods → Java uses strict resolution rules instead of hierarchical traversal
↳ forces classes to resolve conflict explicitly

* Interfaces may be avoided in performance-critical code
↳ involves more dynamic dispatch, limit JVM optimization → introduce small runtime overhead
Though modern JVMs optimize it well, in hot-loops or low latency sys concrete class are still more predictable & faster

* Why default methods were introduced?
problem: [Interface] [default class] → [Class] → conflict → (diamond problem?)
→ To add new methods to interfaces w/o breaking existing code
To provide a default implementation that classes can use or override

Problem b4 J8

Interface Vehicle → (Vehicle: start(); stop();) — take you changed

Vehicle: start();

```
default void stop() {
    S.O.Pln( );
}
```

All existing class implementing Vehicle would break because they must implement stop()
→ backward compatibility problem → large APIs like Java Collection

* Static methods were introduced to keep utility/helper methods related to interface inside the interface
↳ To avoid creating separate utility classes

```
interface MathUtil {
    static int add(int a, int b) {
        return a+b;
    }
}
MathUtil.add(2, 5)
```

Advantage: You can handle many subclasses with one ref type

Parent pc = new Child();

↳ Using a superclass ref limits you to methods defined in superclass.

You cannot directly call subclass specific methods without casting

DOMS | Page No. | Date

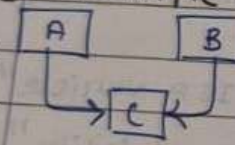
* Types of Inheritance:

1] Single Inheritance: One class extends another class

2] Multilevel Inheritance: A ← B ← C — B extends A, C extends B

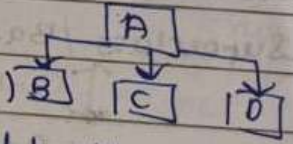
A has no idea of B & C. But C has idea of all above classes A & B

3] Multiple Inheritance: 1 class extending more than 1 class



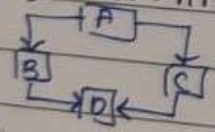
n=5, n=10 (obj = new C();

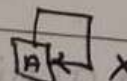
C.n // ? that is why Java doesn't support it



4] Hierarchial Inheritance: 1 class inherited by many class

not in JAVA 5] Hybrid Inheritance: combination of single + multiple inheritance



* You can't inherit yourself.  X → cyclic inheritance

2] Polymorphism: allows same method name or reference to behave differently

* many ways to represent

Q] How Java determines which met

→ For Overloaded methods: matches m

For Overriding methods:

↳ Dynamic Method Dispatch: It is choo over actu

Note: When a method is declared final subclasses to provide a new imple "final prevents method overridi

→ Preserves behaviour, Improve secur

Benefits: Better Performance → when a this m

Because of this guarantee, JVM does n which version of method to call. Inste method call at compile time (call